Model Context Protocol

MCP in depth

What is MCP:

AI Agent → API → if setting changed it will not work

Custom code for API { if API change change the code }

Agent → MCP (MCP Handels API)

AI Agent → MCP request → MCP → API

All Authentical & Authorizat process

V.IMP in depth

V.IMP

MCP

The problem statement:

Newsletter

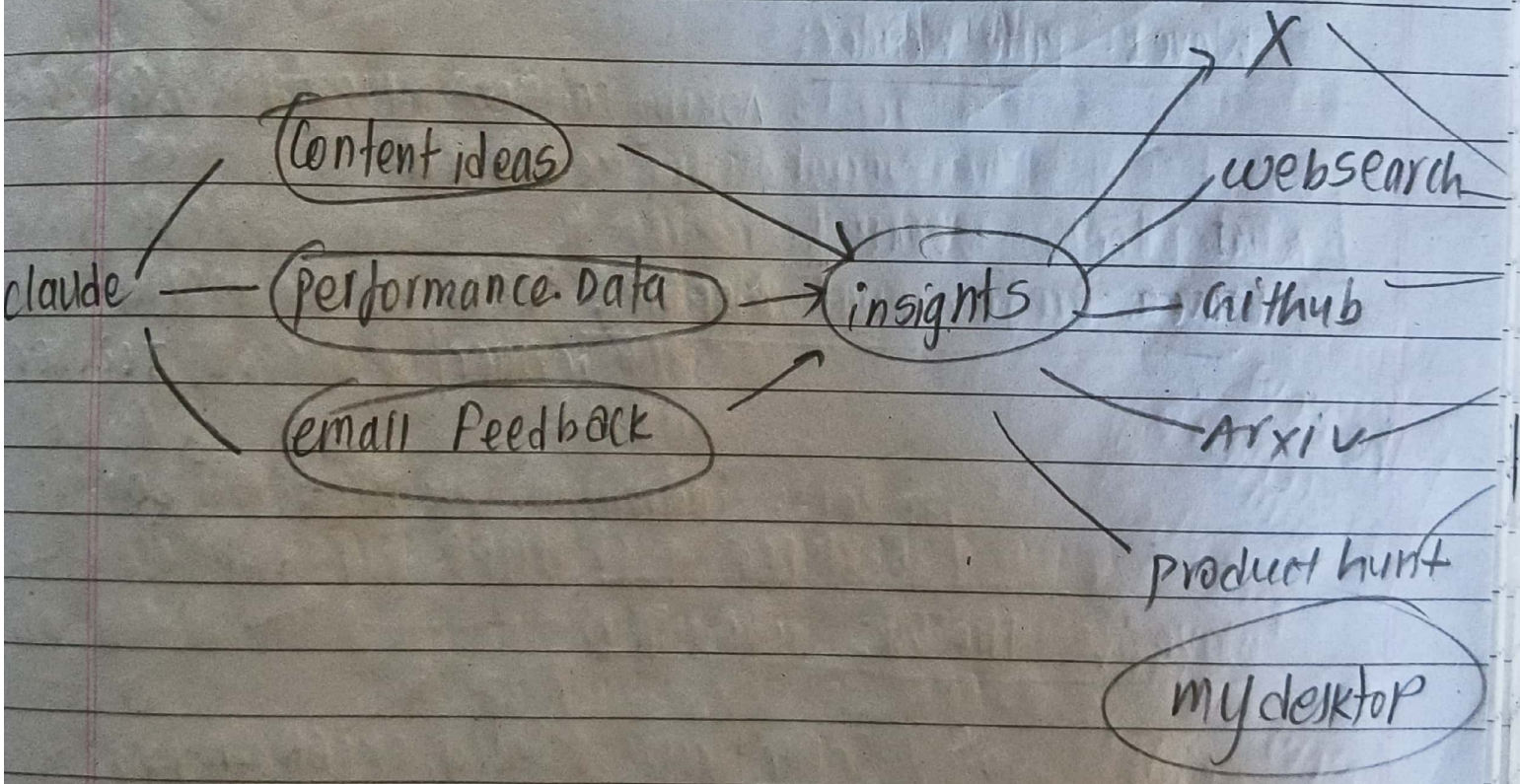
the main AI

claude

the tools

github websearch X

Arxiv product Hint



Scanned with OKEN Scanner

MCP

Part 1 the why with story

30 NOV 2022 → chatgpt released
within 5 days 1 million users
Crossed 100 million users in 2 months.

you can do natural chat with computer.

Waves of Adoption:

Wave 1: Pure Wonder:

Asking random question (Querosity)
Social Media exploded.

Wave 2: Professional Adoption:

lawyer / Developer / Teacher

impact: individual productivity boom.

Wave 3: the API revolution

AI Become more Accessible

Copilot across word, excel ___
AI in gmail ___
first AI tools Cursor

the problem of fragmentation

Notion → AI in notion

Slack → AI in Slack

cannot talk.

tools cannot talk.

the vision vs Reality

We want unified tool/Agent

What is Context:

the Context is everything an AI can see when it generate the response.

formally

context refers to the information that LLm uses to generate answer.

copy paste hell

Context may vary

Context is scattered

for in-context learning.

Needs to paste thousand of line to ask one simple question.

developers have become Human APIs

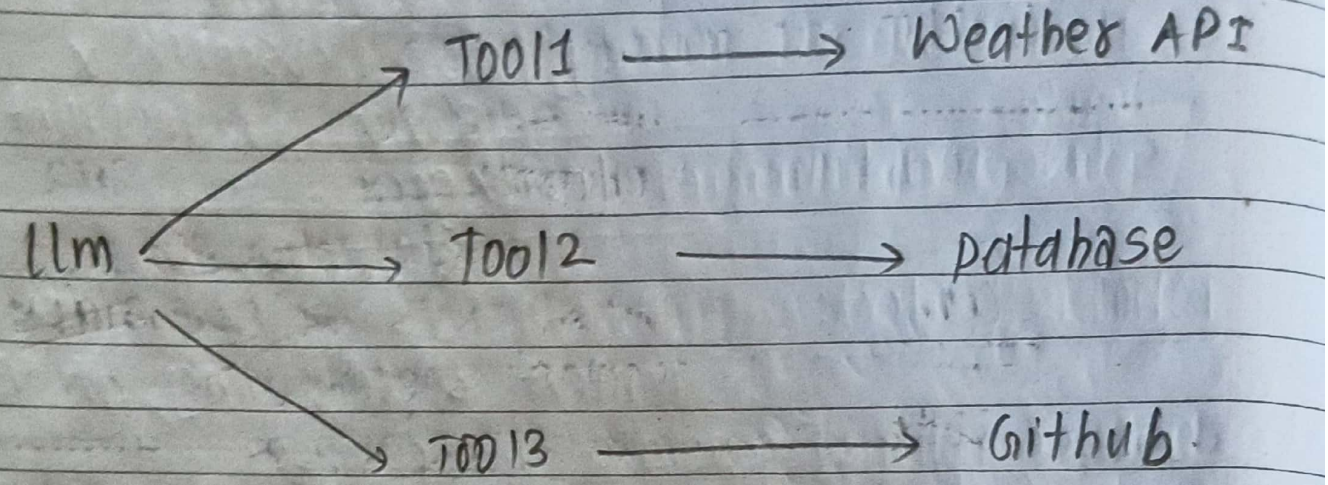
Context Assembly Time > development Time

the solution → function calling

OpenAI introduced function calling in mid 2023

- Function calling is a way using which llms can call ext functions.

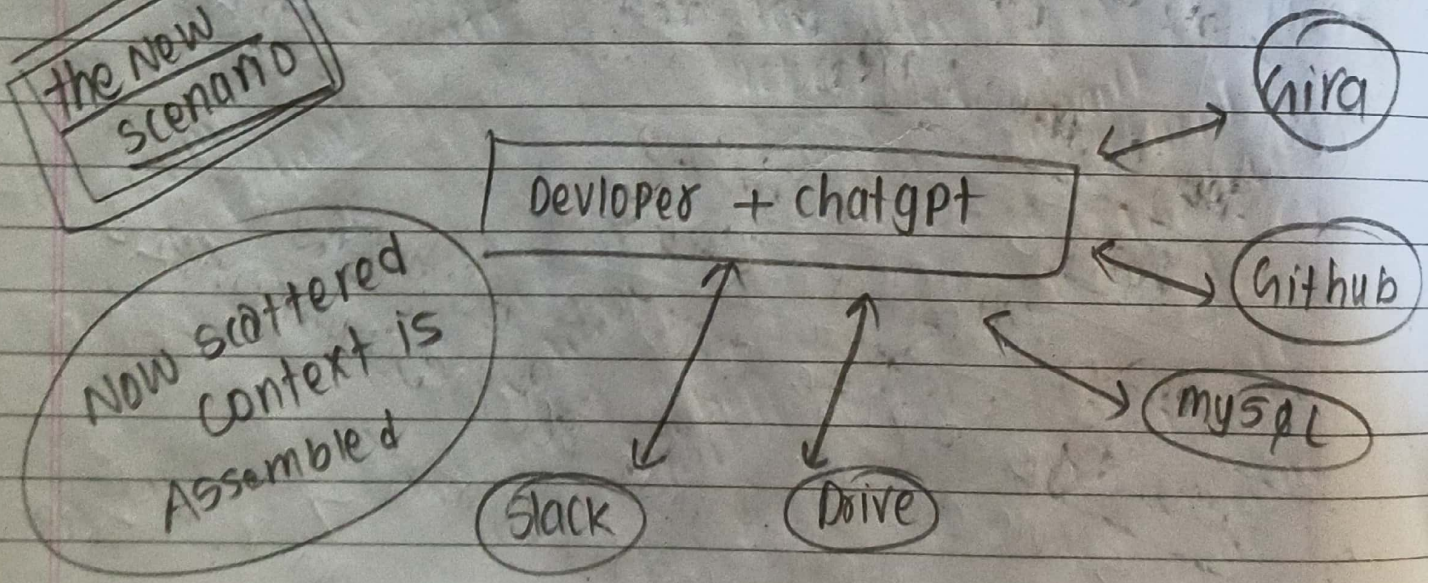
the Rise of tools:



NOW Tools BOOM

A lot of tools in the world.

the New scenario

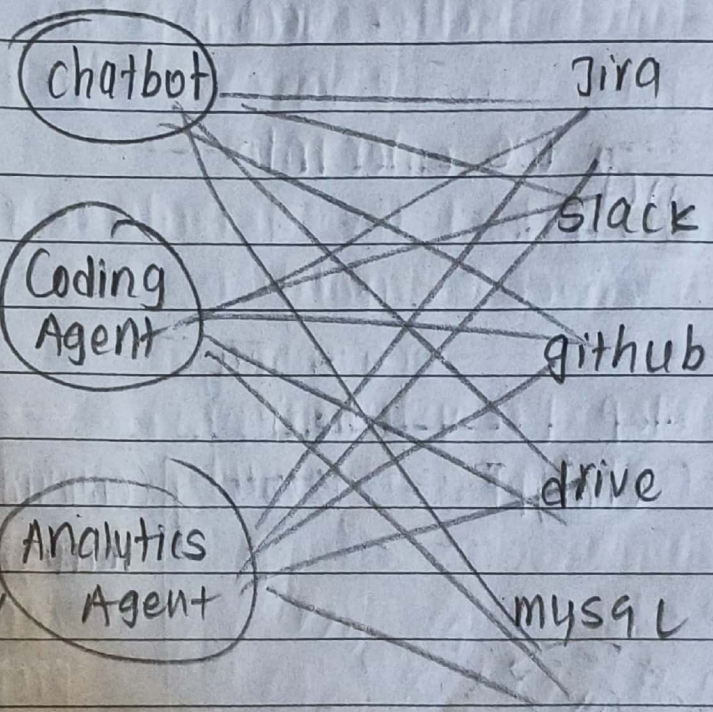


Flow in tool calling

```
@tool
def calculator( first_num: int, second_num: int, op: str)
                                                  ↳ float:
    return {                                    }
```

problem with Tools:

1) integration problem:



NXN

Code Maintainance problem

if google drive API setting change

Google drive Integr fails

2) ~~Hallucination~~ need to debug

3) security Fragmentation

4) Cost & time wastage

overview of the problem

claude
chatgpt
gemini
→ github

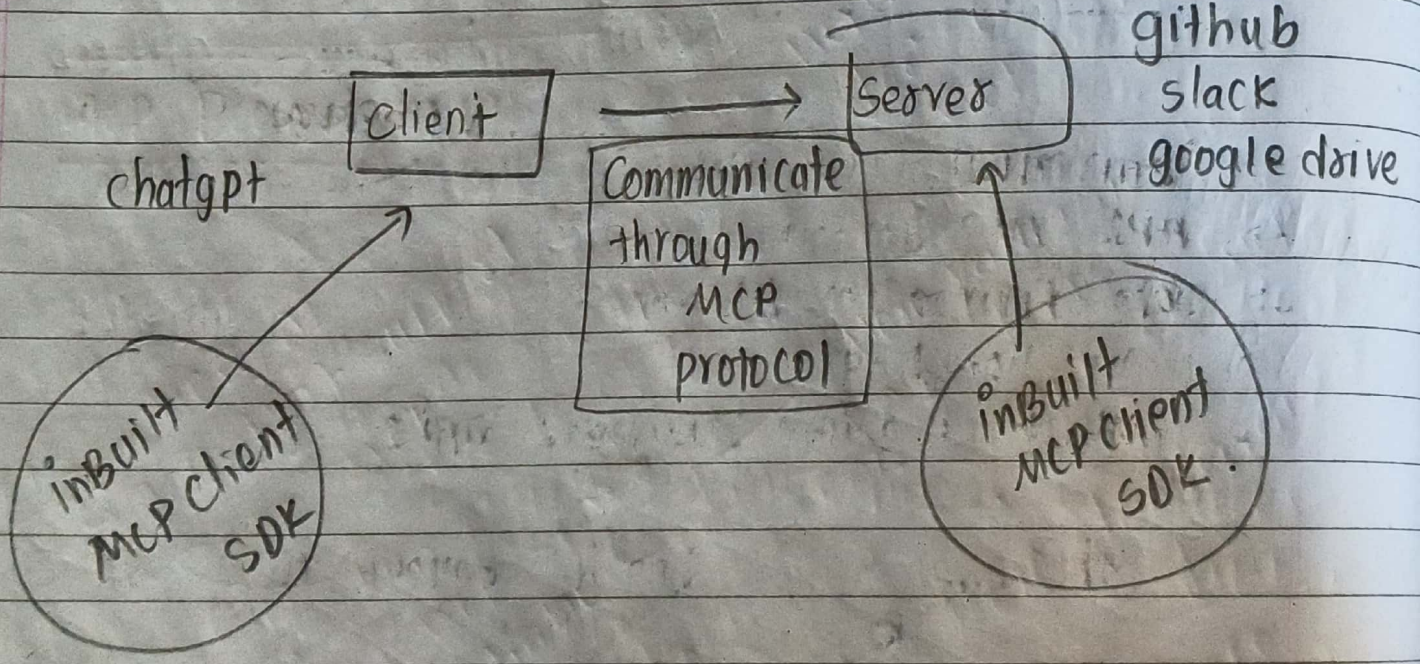
every AI tool was Building its own way to Call every API.

the solution:

one integration For github

Solved By MCP

MCP we enter into



MCP VS Function |Tool calling

Client — Server

Same in MCP

Nothing to do → Just Connect → All heavy lifting done By Server.

function calling

Client
your code ⟷ combine & Worke is done ⟷ Server
company code

Server does the Heavy lifting

Server

- → Authentication with github
- → API rate limiting
- → Data format translation
- → Error handling
- → Github-specific business logic

the client just has to Connect to the server
&
speak same language MCP.

Scanned with OKEN Scanner

Benifits:

N clients & M servers = M+N integrations.

NO Maintanance overhead

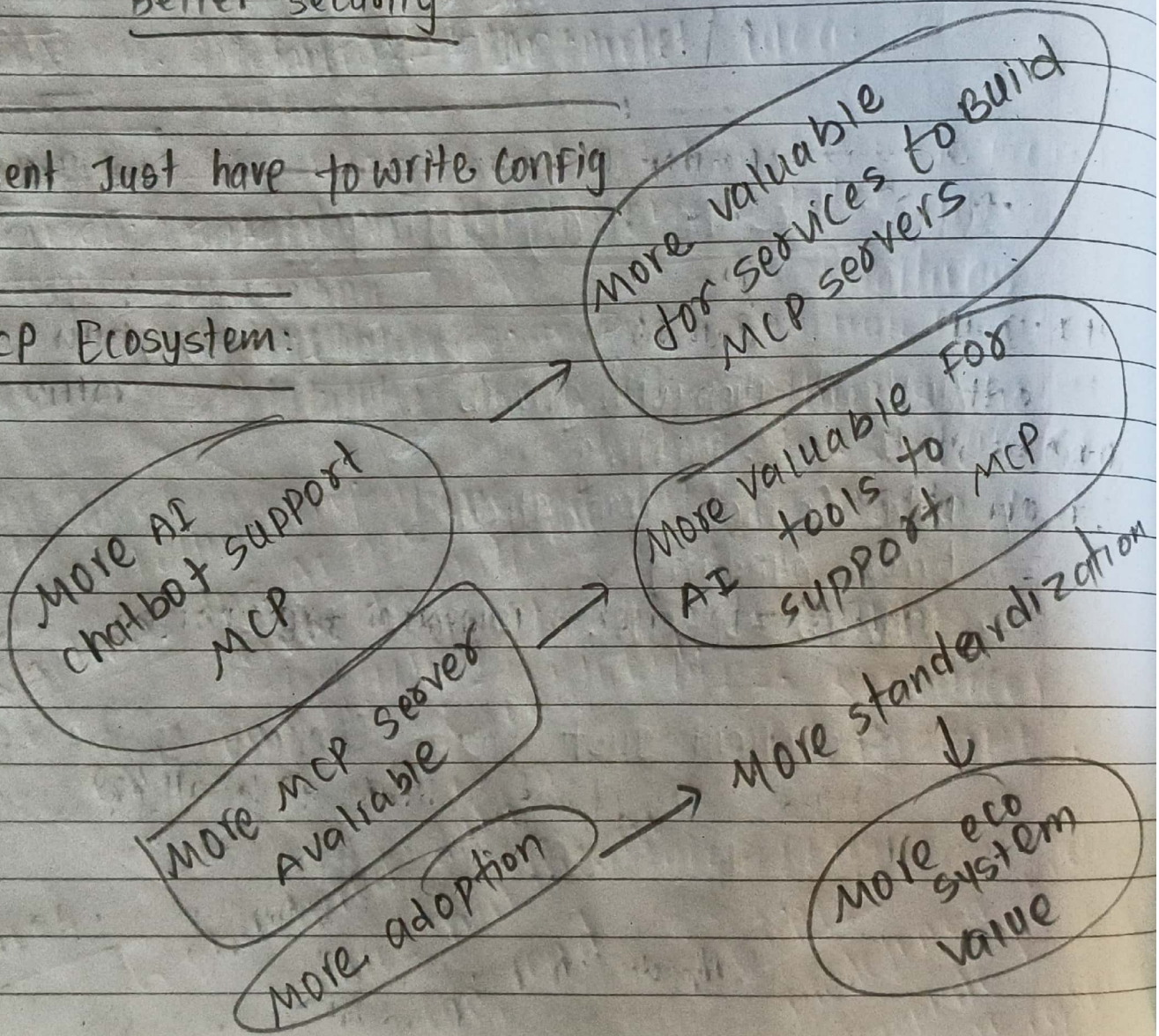
NO problem with Client

{ Reduced cost & Time }

Better security

Client Just have to write config

Mcp Ecosystem:



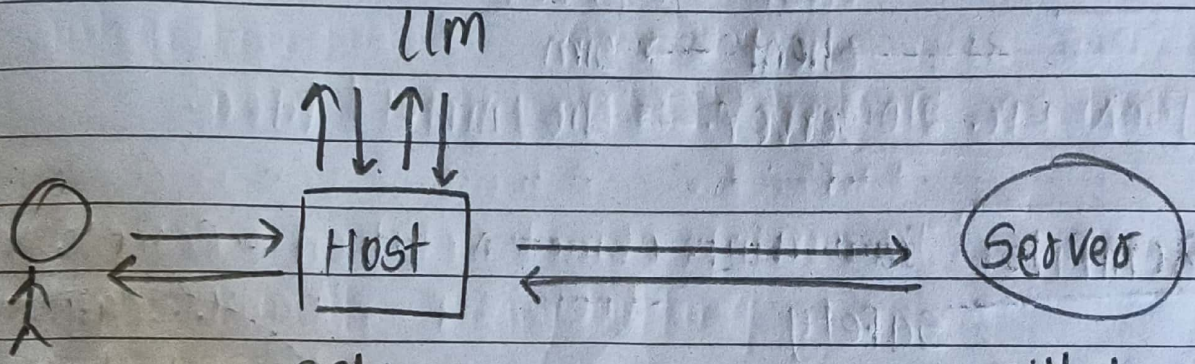
Scanned with OKEN Scanner

Not supporting MCP means cutoff from the rapidly growing ecosystem.

the WHAT

Architecture | MCP lifecycle | Advance concept

Architecture

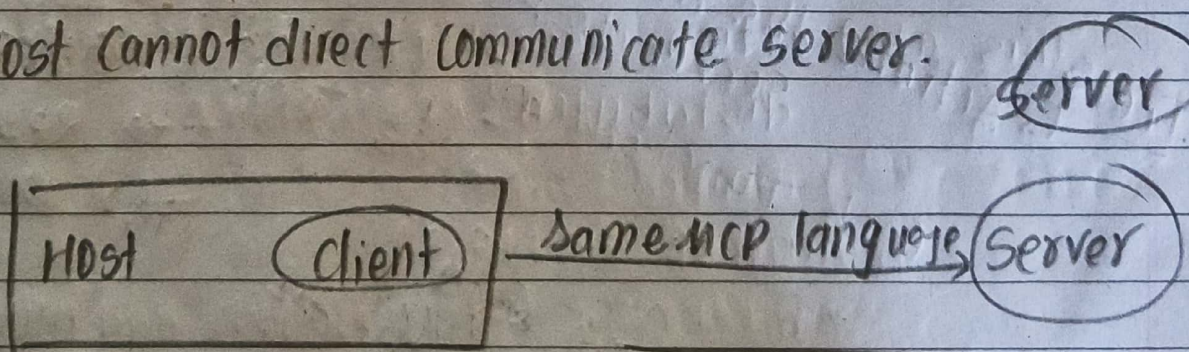


are they any new commits in the new repo

gpt
Claude
gemini

github
Slack
brive

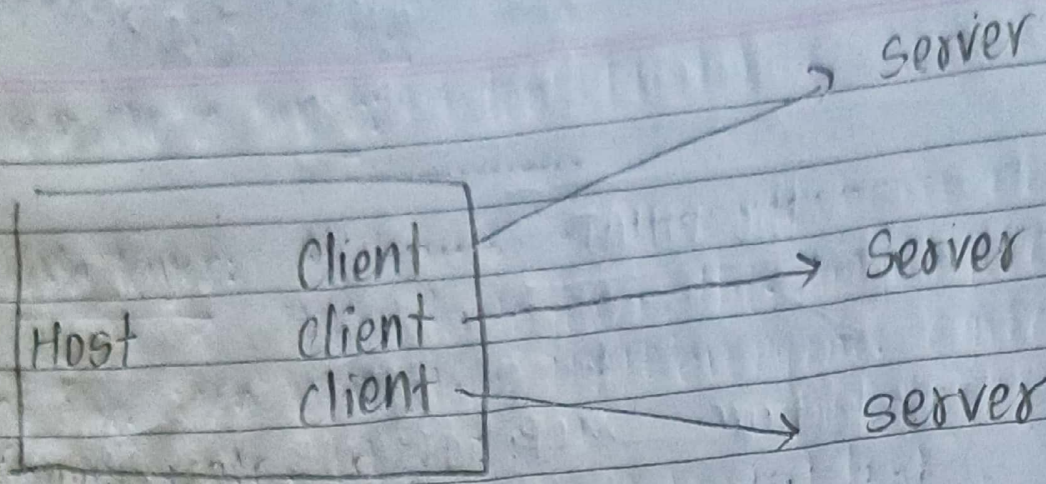
★ Host cannot direct communicate server.



one on one relation

Client can only communicate to only one server.

Scanned with OKEN Scanner



Analogy

Host → phone
client → sim
Server → Jio / Airtel / idea.

Benifits:

safety
Decoupling
Parallelism
Scalability

primitives ?

things the Server Can OFFer to Host.

- Tools
- Resources
- prompts.

Tools
↳ Actions the AI ask the server to platform
each functionality
↳ one Tool.

Resources:
↳ structured data source that the AI Can read

Prompts:
predefined prompt template to shape AI respons

88

primitives - standard operations

Tools

tools/list → list of All. tools Server prov-
tools/call → run this tool with Args.

Resources:

resources/list → list of all static docs
resources/read → Content of this res
resources/subscribe/unsubscribe → Changes notifica

prompts:

prompts/list → all prompt template
prompts/get

MCP Data layer:

Data layer:

the data layer is the language and grammar of the MCP ecosystem that everyone agrees upon to communicate.

in MCP JSON RPC 2.0 serves as the foundation of the data layer.

JSON RPC 2.0

"Javascript object Notation - Remote procedure call"

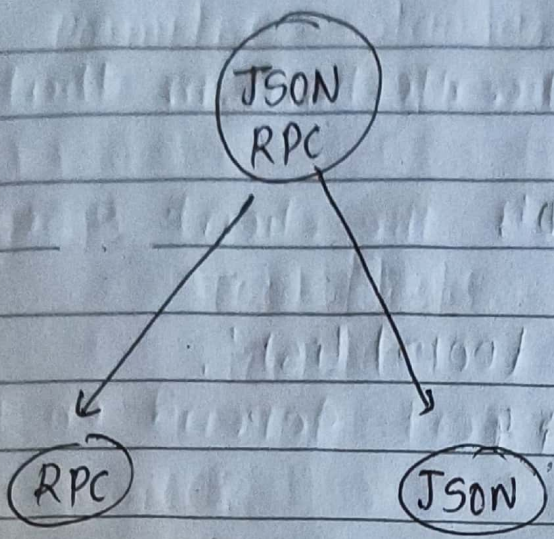
RPC allows a program to execute a function on another computer as if it were local, hiding the details of network communication and data transfer.

This abstraction makes it easier to build distributed Applications.

instead of add(2,3)

↳ please run add with parameters 2 & 3.

JSON RPC:



JSON-RPC Combines the concept of Remote procedural calls with the simplicity of JSON.

Allowing developers to structure RPC requests and response in a standardized JSON format.

JSON RPC 2.0

Request structure:

```
{ 'JSONRPC' : '2.0'
  'method' : 'add',
  'Params' : [2,3],
   'id'    : 1
}
```

Response structure

```
{
 'JSONRPC' : '2.0'
 'result' : 5.0'
  'id'  : 1
}
```

```
{ JSONRPC : 2.0
  error: { 'code' : -32601
  'message' : 'method not
              found'.
  }
  id : 1
}.
```

1) Discover tools (tools/list)

request:

```
{
  'jsonrpc' : '2.0',
  'id'      : 1,
  'method'  : 'tools/list',
  'params'  : {}
}
```

response:

```
{
  'jsonrpc': '2.0',
  'id' : 1
  'result': {
              'tools' : [ {'name': 'github.listissu
                           'description': '
                          },
                        ]
            }
}
```

(2)

Call a tool (tools/call) → list issues:

```
{
  'jsonrpc' : '2.0'
  'id' : 2
  'method' : 'tools/call',
  'params' : {
               'name' : 'github-list-issues',
               'arguments': {'owner': 'ownerName'}
```

Scanned with OKEN Scanner

MCP
↓
FASTAPI

response:

```
{
 'jsonrpc': '2.0',
  'id': 2
  'result': {
      'content': [
          { 'id': 271, 'title': 'Crash on startup',
          'state': 'open' },
          ]
  }
}
```

in notification
↳ no id
& no response

why JSON-RPC for Data layer ??

1) its lightweight. (no header + no metadata)

2) JSON-RPC support bi-directional communication.

3) it is transport - agnostic.
(can support http, websocket, custom

4) support batching.

5) supports notification.

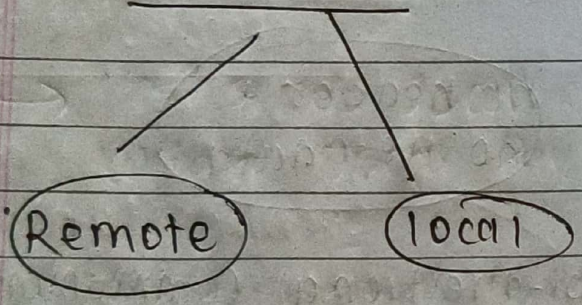
Scanned with OKEN Scanner

MCP Transport layer:

The Transport layer is the mechanism that move JSON-RPC messages betn the client & server.

the choice of the Transport depend on the type of server.

MCP Types of Servers



(STDIO)

A local server is a program running on your own comput

A remote server is a program running on another computer (somewhere else on the NIW or internet)

that you connect to over a network.

HTTP | SSE

local Servers: (STDIO)

STDIO – refers to the built in streams every program has.

stdin (input the program reads).
stdout (output the program writes)

in MCP these streams are used as transport layer between the client & server.

HOW STDIO WORKS:

→ the host launches the server as a subprocess on the same machine.

→ the host (client) writes JSON-RPC message into the server's STDIN.

→ The server reads those messages, processes them & writes back responses to it's STDOUT.

Scanned with OKEN Scanner

Benefits of the STDIO transport:

→ Fast
→ secure.
→ very simple to implement.

V.IMP

Remote Servers — HTTP + SSE

→ Using HTTP allows the host to reach servers running anywhere.

→ Host sends JSON RPC requests using POST requests with a JSON Payload.

→ the transport supports standard HTTP auth methods (like API keys.).

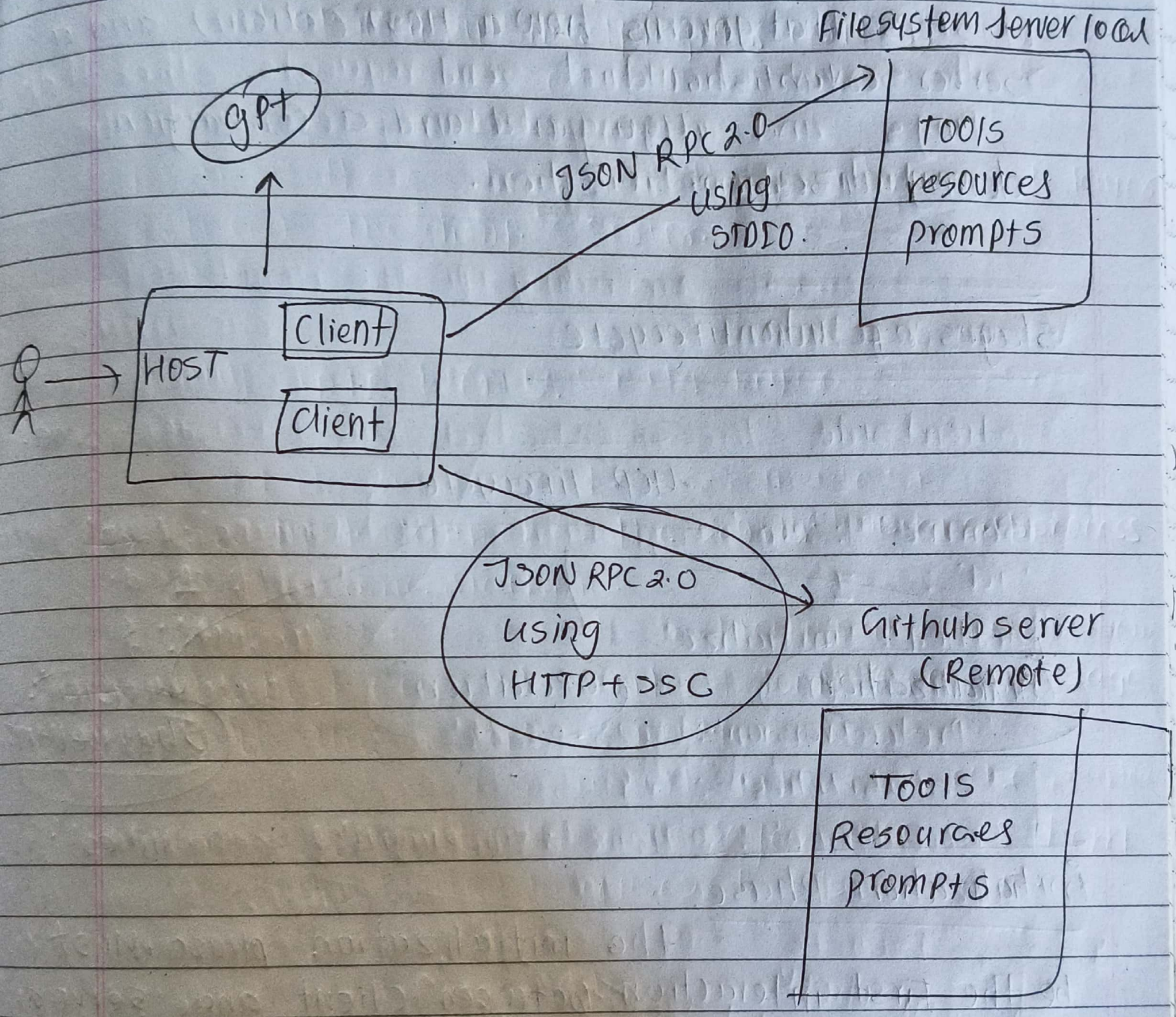
SSE → Server sent Events: (used for streaming)

Using SSE the server sends multiple messages to client over a single open connection.

Scanned with OKEN Scanner

Instead of sending one large, JSON blob the server can stream chunks of data as they are ~~redy~~ ready.

Ideal for long running tasks or incremental updates.



Scanned with OKEN Scanner

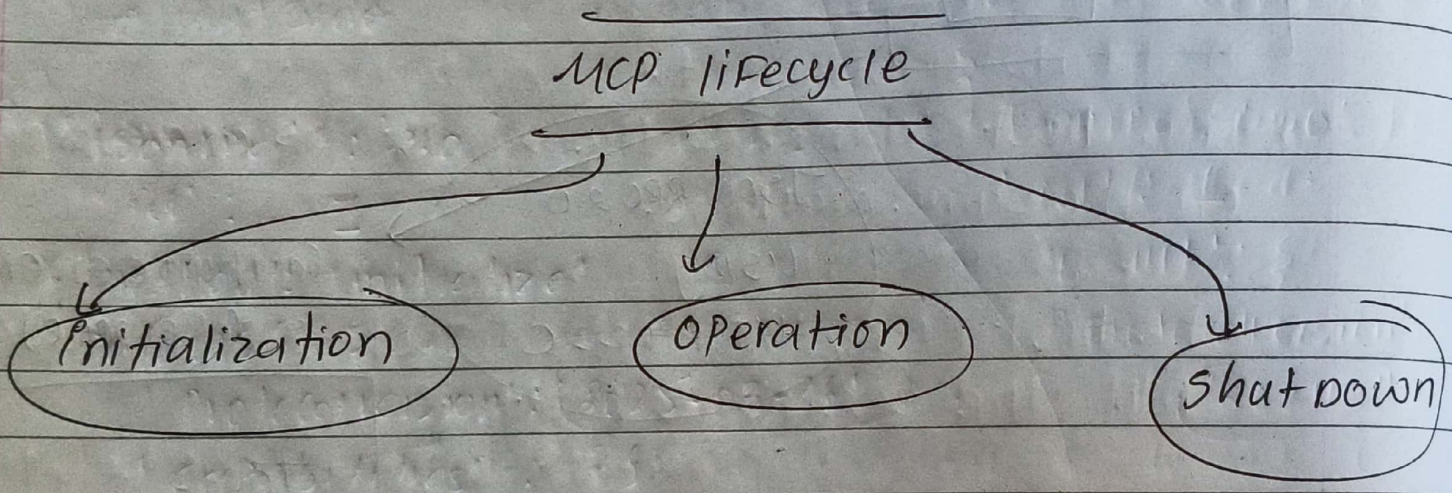
V.IMP

MCP lifecycle

what is MCP lifecycle:

the MCP lifecycle describes the complete sequence of steps that governs how a HOST (client) and a server establish, use and end a connection during a session.

Stages OF MCP lifecycle



Initialization phase:

the initialization phase must be the first interaction between client and server.

- establish protocol version compatability
- exchange and negotiate capabilities.

Scanned with OKEN Scanner

Initialization phase 1:

the client sends a initialize request containing.

1. MCP protocol version
2. client capabilities.
3. client implementation info.

Initialization phase 2:

The server sends it's own capabilities and info.

1. MCP protcol version.
2. server capabilities.
3. Server implementation info.

Initialization phase 3

After successful initialization, the client must send an initialized notification to indicate it is ready to begin normal operations.

```
{ 'jsonrpc' : '2.0',
  'method' : 'notifications/initialized'
}
```

Now the client & server are connected.

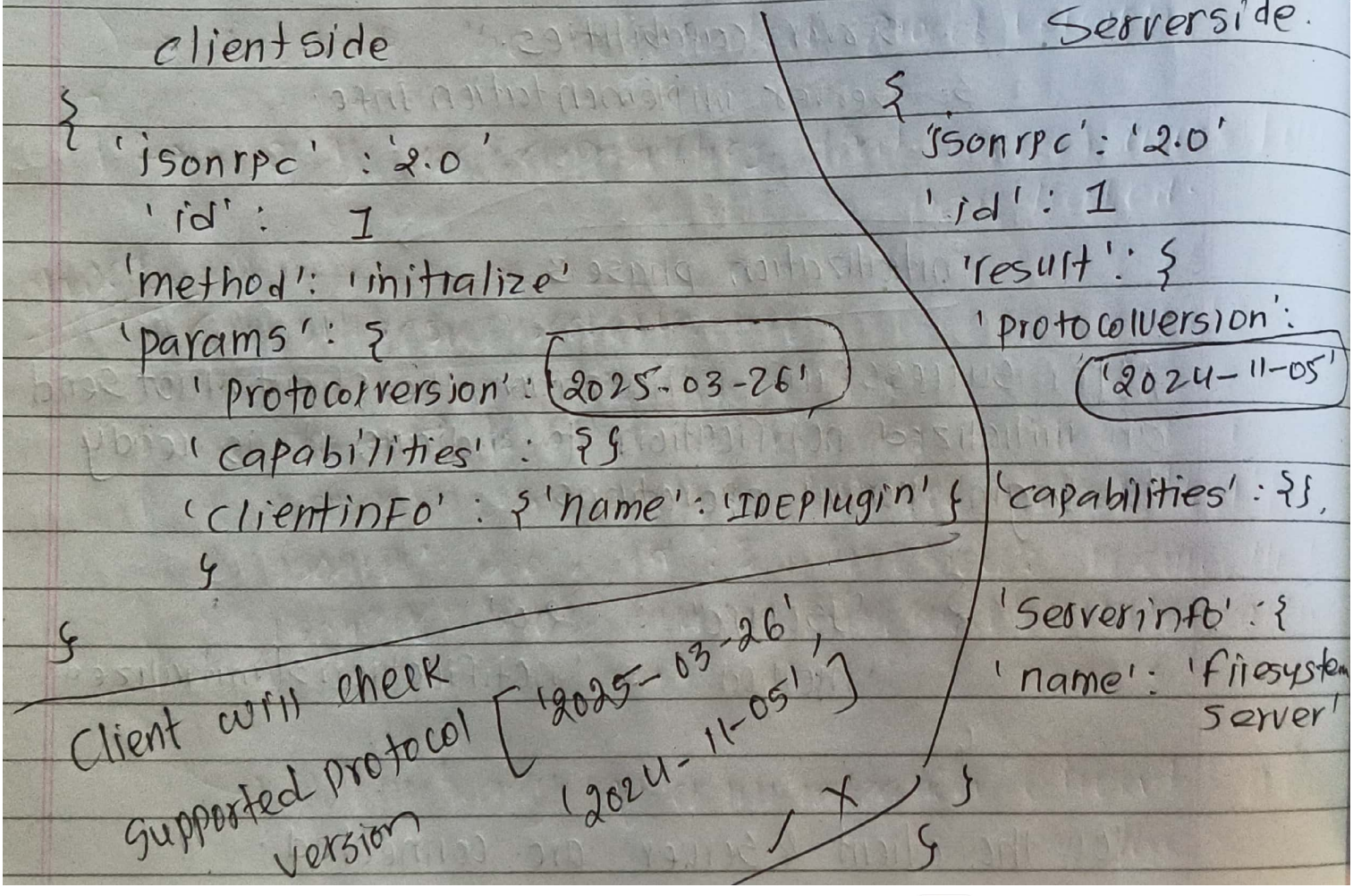
Scanned with OKEN Scanner

important rules:

⇒ the client should not send requests other than pings before the server has responded to the initialize request.

⇒ the server shouldnot send requests other than pings and logging before receiving the initialized notification.

Version negotiation



capabilities Negotiation:

client and Server capabilities establish which protocol features will be avaliable during the session.

client → roots.
client → sampling
@t
client → elicitation.

Server → Tools
server → Resources
Server → Prompts
Server → logging.

Sub Capabilities:

listchanged:
changes in server

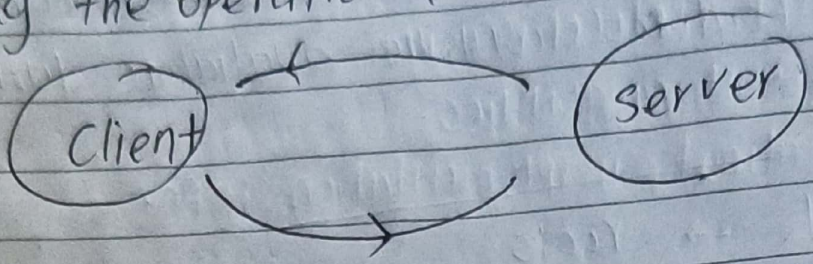
Subscribed:
Any change in resources.

Scanned with OKEN Scanner

step2

operation phase:

During the operation phase:



exchange messages

↓

According to the negotiated capabilities.

- → Respect the negotiated procol version.
- → only use capabilities that were successfully negotiated.

Phase 1

capabilities Discovery:

→ Batch request

```
[ {'jsonrpc': '2.0', 'id': 2, 'method': 'tools/list'}
  {'jsonrpc': '2.0', 'id': 3, 'method': 'resourcellist'}
  {'jsonrpc': '2.0', 'id': 4, 'method': 'promptlist
]
```

→ Automatically After initialization Client send this.

Scanned with OKEN Scanner

Capability Discovery:

list → Client ↔ host.

Phase2

Tool calling

Steps

Shutdown phase:

one side (Typically the client) initiates shutdown

No special JSON RPC shutdown Message is defined

Transport layer is responsible For signaling Termination.

Shutdown in STDIO:

Client initias shutdown (should):

1) Client close the input stream to the child Process (server)
2) wait For server to exit.
3) send SIGTERM if server doesnot exit in time
4) send SIGKILL if still unresponsive (with help of os)

Scanned with OKEN Scanner

Server initiated shutdown (May should):

→ close output stream to the client
→ exit process.

SHUTDOWN in HTTP:

Client-initiated shutdown (common case):

↳ The client (host) close the HTTP connection(s) ↳ it opened for the server.

Server initiated shutdown (possible)

→ the server may close the connection from its side

→ The client must be prepared to detect a dropped connection and handel it.

(eg: reconnect if appropriates.

Scanned with OKEN Scanner

No JSON RPC Message For the shutdown:

special cases

Pings:

Ping is a light weight request/response method defined in MCP.

Purpose:

to check whether the other side (most or server) is still alive and the connection is responsive.

Request:	Response:
{ 'jsonrpc': '2.0', 'id' : 42, 'Method' : 'Ping' }	{ 'jsonrpc' : '2.0', 'id' : 42, 'result' : {} }

When ping used?

1. useful for checking if the other side is up before Full initialize.
2. if no Activity, OS, proxies, firewall drop the conn. So, client send periodic pings.

Scanned with OKEN Scanner

Error Handling

Error handling in MCP is

↓

How Host (client) and server ~~signal~~

⇓

signal

⇓

If something went wrong with a request.

Mcp inherits JSON rpc standard error object format.

Causes of Error:

1) unsupported or mismatched protcol version.
2) Calling a method for a capability that wasn't negotiated.
3) invalid arguments to a tool.
4) internal server failure while processing a request
5) Timeout exceeded → Client Cancels request.
6) Malformed JsoN-RPC messages.

Scanned with OKEN Scanner

Error object structure:

```
{
  'jsonrpc' : '2.0' ,
  'id'      : 5,
  'error'   : {
      'code'    : -32601,
      'message' : 'Method not found',
      'data'    : {
          'extra' : 'optional info'
      }
  }
}
```

ode

teger code
that catagorize
the error

message
↳ short human-readable explanation

data
↓
optional
↓
extra structured data
for debugging
or
context.

Scanned with OKEN Scanner

Common Error Codes:

- -32601 → Method not Found → Method exists
- -32602 → Invalid Params → Missing or wrong Params
- -32600 → Invalid request → Malformed JSON-RPC request structure
- -32700 → Parse error → JSON could not be Parsed.
- -32000 and above → server defined errors → Authentication Failure, rate limit exceeded, quota errors internal issues.

Timeout

V. IMP

Timeout

* Timeout is about ensuring request's don't hang Foreever.

Purpose:

↳ Protects against unresponsive or overloaded servers.

↳ Ensures resources (Memory, CPU) aren't held indefinitely.

↳ gives the user feedback instead of waiting foreever.

How Timeout works:

* SDK let client sets a per request timeout (eg: 30s)

* if the deadline passes with no result ↩ Client triggers a timeout.

* Client then sends an cancellation notification to tell the server to stop.

* the server must stop processing that request and not return a result.

Scanned with OKEN Scanner

Cancellation notification:

request

```
{
  'jsonrpc': 2.0
  id : 7
  method: tools/call
  params : {
            name : searchCode
            arguments : { query: MCP },
            '_meta : { progress token: tok-
           }
}
```

response

```
{ jsonrpc: 2.0
  method: notifications/cancelled
  params :
          {
            requestid: 7,
            reason : Timeout exceeded (30s)
          }
}
```

Scanned with OKEN Scanner

Progress Notification

purpose:

let the client know a long-running request is still making progress.

client includes a progressToken in the request's _meta.

Server can then send notifications/progress update while working.

client request:

```
{ 'jsonrpc': 2.0
  id : 7
  method : tools/call
  Params : {
      name: searchCode
      arguments : {query : MCP lifecycle}
      _meta : { progressToken : tok-7 }
}
```

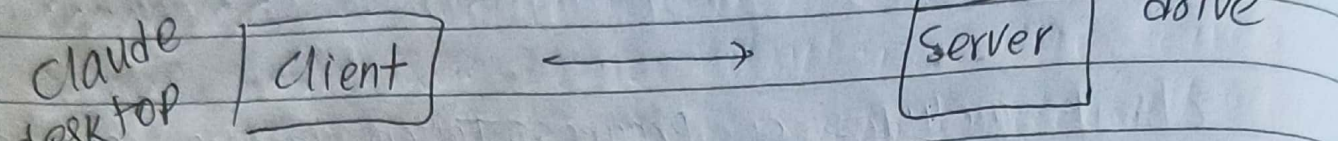
Server response

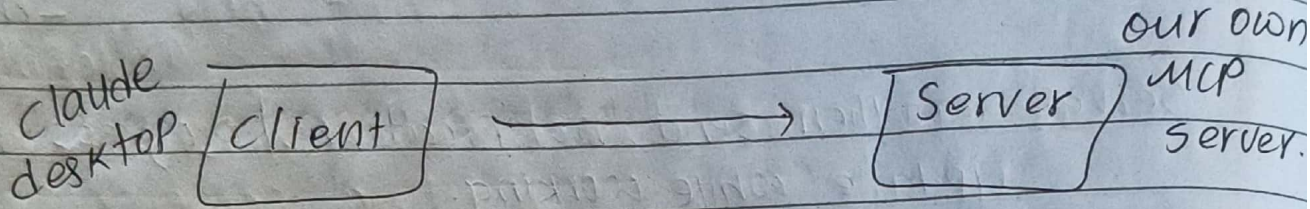
```
{ jsonrpc: 2.0,
  method: notifications/progress
  params: { progresstoken : tok-7
            progress: 60
            total : 100, Message: 'searching fil
```

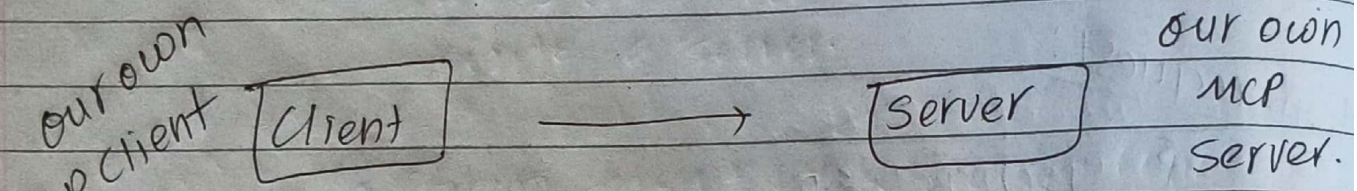
Coding part practical MCP

the HOW?

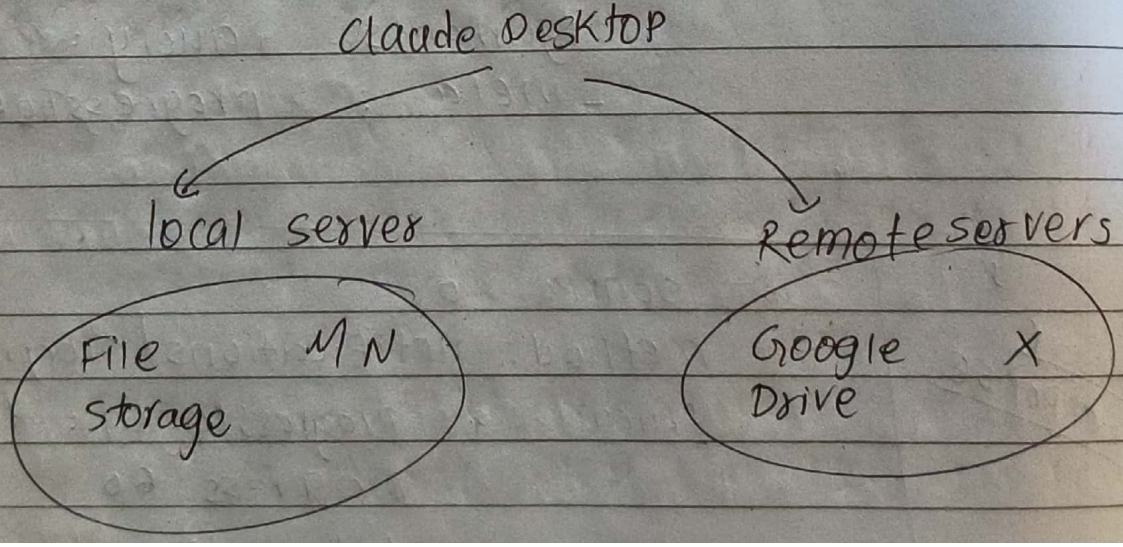
Strategy:







Plan OF Action:



Scanned with OKEN Scanner

Types of Connections:

1. Connectors: 2. Using config files

What are Connectors?

A connector is a built-in feature that links claude to MCP servers automatically, without the need for manual setup or configuration.

V. Usefull for non-technical end users

direct connectors:

easy, safe & consistent.

the connector system wraps an MCP server behind the scenes and handles authentication via oAuth

think of Connectors as the App store for MCP servers.

User-friendly
Click-based
pre-curated.

Why not use Connectors always?

Connectors are curated & Managed.

Connectors are officially built, hosted, and maintained by Anthropic.

Scanned with OKEN Scanner

→ They come with oAuth login flows, managed security rate-limits and guranteed stability.

→ If every MCP server were required to be a connector, it would mean Anthropic has to review, host and secure every possible server. which doesn't scale.

→ MCP is an open standard.

↳ MCP is designed so anyone can write a server.

↳ Forcing everything through connectors would close the system (ecosystem) and make you dependent on Anthropic to approve ~~the~~ or public servers.

Scanned with OKEN Scanner

3Blue1Brown

local server

Filesystem & Manim-mcp-server

for visualization:

Remote servers:

google Drive

twitter

{weather MCP server}

Awesome MCP servers:

* How to Build local mcp servers.
  How to Build & deploy Remote MCP servers.
  How to Build MCP clients.

* Building local MCP servers:

↳ Intermediate MCP server.

↓

expense Tracker MCP server:

through Natural ~~cmds~~ language

power tools

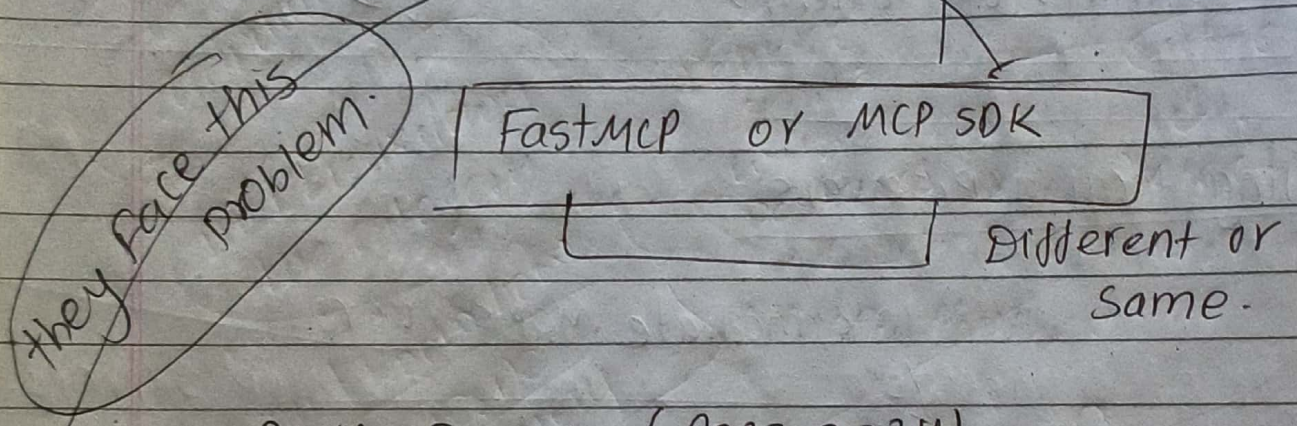
very good approch

useful MCP server

demo server (calculator)
→ install
→ run → claude

incremental → expense Tracker → local server → STDIO
↓
improve → remote server → host.

Mcp → protocol → set OF rules
→ Complex
→ redundant



1. Birth OF Mcp (2023-2024)

→ Anthropic introduces Model Context protocol (MCP) &structured Communication between LLMs & external tools/ data

→ Early needs: → Build MCP servers (tools/ reso/ pr
↘ Build MCP clients. (Apps).

Scanned with OKEN Scanner

Official Python SDK (mcp) released:

→ mcp.servers → Build servers.
→ mcp.clients → Build clients
→ mcp.cli → CLI for debugging / testing
↳ pip install "mcp[cli]"

2. Developer Friction & FastMCP 1.x (2024)

Problem: Raw SDK = verbose & boilerplate heavy

Jeremiah Lowin → Prefect CEO →
FastMCP on top of mcp SDK.

mcp.server.fastmcp

Popular for teaching & prototyping.

(3) FastMCP Breakout (2025)

* FastMCP adoption grows rapidly

* FastMCP 2.x released as standalone package

Scanned with OKEN Scanner

Day3 MCP
very important

Sam Altman

cheik with MCP inspector

expense Tracker

Tryout this one

1) Add expense
2) list expense
3) summarize
4) ~~Edit~~ expense
5) delete expense
6) credit Salary

Code Manually

just breakdown the category

just knocs how to write query

replace sqlite with postgres

Database
↓
expenses
↓
sqlite
↓
projectFolder

Add Resource

categories.json

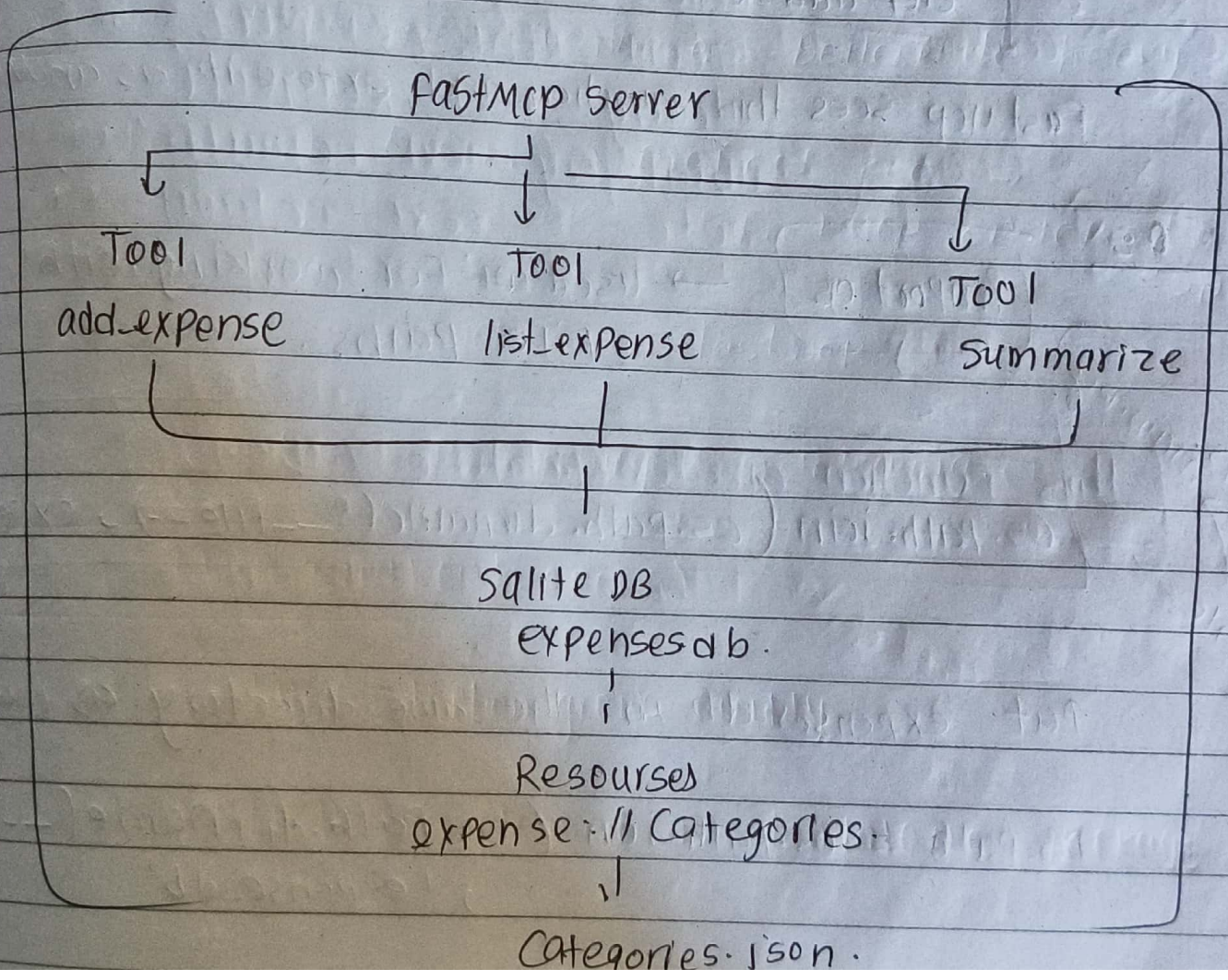
Scanned with OKEN Scanner

FastAPI Compatability with FastMCP

Convert FastAPI server into Fastmcp:

git pull --rebase origin main
git push -u origin main

main.py



Scanned with OKEN Scanner

FastMCP exposes the python - Functions as MCP tools / Resources.

step 1 imports

while sqlite store the actual expense data.

① FastMCP

```
From fastmcp import FastMCP
```

→ FastMCP Framework

→ Turn normal python Func to MCP To

eg

```
@mcp.tool()
def add_expense(...):
```

FastMCP sees this function and expose it as an MCP too

② os

```
import os
```

→ used ~~of~~ For working with File paths.

refer main.py __file__

```
os.path.join(os.path.dirname(__file__), expense.dp)
```

put expenses.db in the same directory as main.py.

```
DB_path = os.path.join(os.path.dirname(__file__)
                       , 'expense_db'
```

Scanned with OKEN Scanner

④ sqlite3

import sqlite3 → python builtin

```
categories_path = os.path.join(
    os.path.dirname(__File__),
    'categories.json'
)
```

Now you have 2 data sources

expense.db ⟶ actual expense records
categories.json ⟶ available categories.

Creating the Mcp server

```
mcp = FastMCP('name = 'ExpenseTracker')
```

mcp server object →

we will later attach tools and resources to this object:

```
@mcp.tool()
```

with this decorator.

Scanned with OKEN Scanner

(5.) Database Initialization:

if expenses.db doesn't exists sqlite create it

opens sqlite database

```
def init_db():
    with sqlite3.connect(DB_Path) as c:
        c.execute("""
            CREATE TABLE IF NOT Exists expenses(
                id INTEGER PRIMARY key Autoincre
                date Text NOTNULL,
                amount REAL NOT NULL,
                category Text NOT NULL
                subCategory Text Default '',
                note Text Default ''
            )
        """)
```

create the table expense if not exist

this function creates database Table.

→ with sqlite3.connect(DB_PATH) as c:

↳ this opens the database connection and give you a connection called c.

the with statement also helps handle closing/committing the connection properly.

init_db()

main.py starts.

↓

init_db()

↓

open expenses.db

↓

creates expenses table if needed.

↓

Continue starting server.

---

② First MCP Tool - (add_expense)

```
@mcp.tool()
def add_expense(date, amount, category,
                subcategory="", note=""):
```

Scanned with OKEN Scanner

(10) Parameters:

```
def add_expense(
        date,
        amount,
        category,
        subcategory = "",
        note = ""
):
```

last two have default.

~~So~~

(11) insert into sqlite:

```
with sqlite3.connect(DB_PATH) as c:

    cur = c.execute(
    "
    Insert into expenses (date, amount,
                          category, subcategory
                          ,note)
                values( ?, ?, ?, ?, ? ),
```

placeholders replaced by → (date, amount, category, subcategory, note).

```
    )
```

4 4 4 4

Both
FastAPI
FastMCP

② why use ?

F"""
insert into expenses values ('{date}', '{amount}',)
"""

we use parameterized SQL.

③ Return value:

return {
'status': 'ok'
'id': cur.lastrowid
}

How to Build & deploy Remote MCP servers

local MCP server ✓
↳ Fast

FastMCP
↳ Remote MCP server ✗
↳ multiple clients
↳ slow → over internet

diff on internet server
↳ powerful

enterprise MCP servers

① Simple remote server ⟶ easy Code POV
same as local

② expenseTracker ⟶ Remote

③ deploy ⟶ FastMCP cloud.

④ Problems ⟶ rectify

Terminal 1

FastMCP
↓
0.0.0.0:8000
↓
/mcp

Terminal 2

mcp inspector
↓
streamable HTTP
↓
http://127.0.0.1:8000/mcp

instead of LH.

Flaws
1) synchronous
Fix
async await
aiosqlite

add aiosqlite

claudecode hack
to add unlimited custom connector

proxy server

Claude desktop

proxy

Remote mcp server

local mcp server

Sampling

elicitation

Authentication

v.imp @100

Building mcp clients

chatbot

math

expense tracking

local server

Remote server

→ local connect (math)
→ expense Remote
→ UI (streamlit)

MCP servers

Personal chatbot

Scanned with OKEN Scanner

The More you learn the lesser it will take to learn even more

SR

MCP-Math-server

done Add local MCP server

→ install uv
→ create on Folder
↳ uv init
↳ uv add ~~fast~~mcp

→ create main.py

Make a client connect to local server

→ pip install uv

→ create Folder youtube_MCP_client

→ uv init .

→ uv add langchain langchain-openai
langchain-mcp-adapters python-dotenv
streamlit

Scanned with OKEN Scanner

client1.py version-1 Async await

import

do this → After

Tool Node / Tool Condition integrated

MCPClient LangGraph

client1.py

imports → import asyncio

Because MCP communication & LLM calls are asynchronous.

async def main() await client.get_tools()

From langchain_mcp_adapters.client import MultiServerMCPClient.

.dumps()
convert into json text

Server = {

'math' : {

'transport' : 'stdio',

'command' : 'uv path',

'args' : [

'run',

'Fastmcp',

'run

'path of main.py'

],

},

you are telling it to execute uv.

equivalent uv run fastmcp run path of main.py

transport → stdio → Client

stdin/stdout

giving our MCP client authority to launch our math server

MCP server

the MCP server runs as a local process.

instead of starting a local python process. your client connect to. using

expense : {

'transport' : 'streamable-http',

'url' : _____,

},

streamable_http

our python program

HTTPS

FastMCP cloud

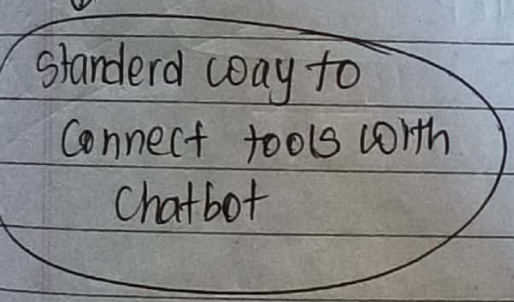
expense MCP server

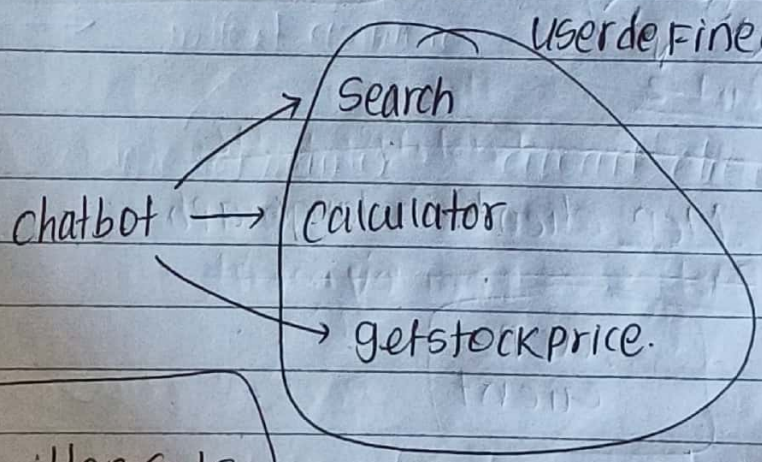
cd "$Home\Desktop"

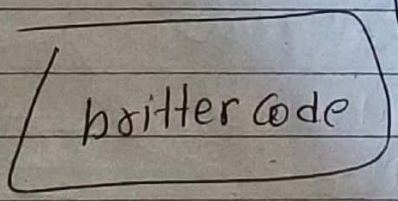
* langgraph MCP client

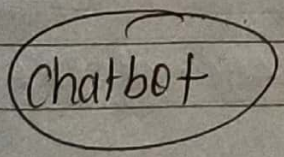
Recap — chatbot with MCP langgraph

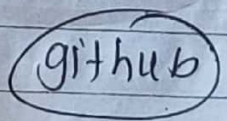
mcp → improved version → TOOLS
↳ flaw
Solved by MCP











tool → api 1.0 → api 2.0

Change in code

Very Import Flaco

Manually update the code for tool:

changes made on server should not affect client

Only config code in client

All tools in server side

Maintaince problem solved By MCP

inclined toward mcp

Mcp client in langGraph

mcp client

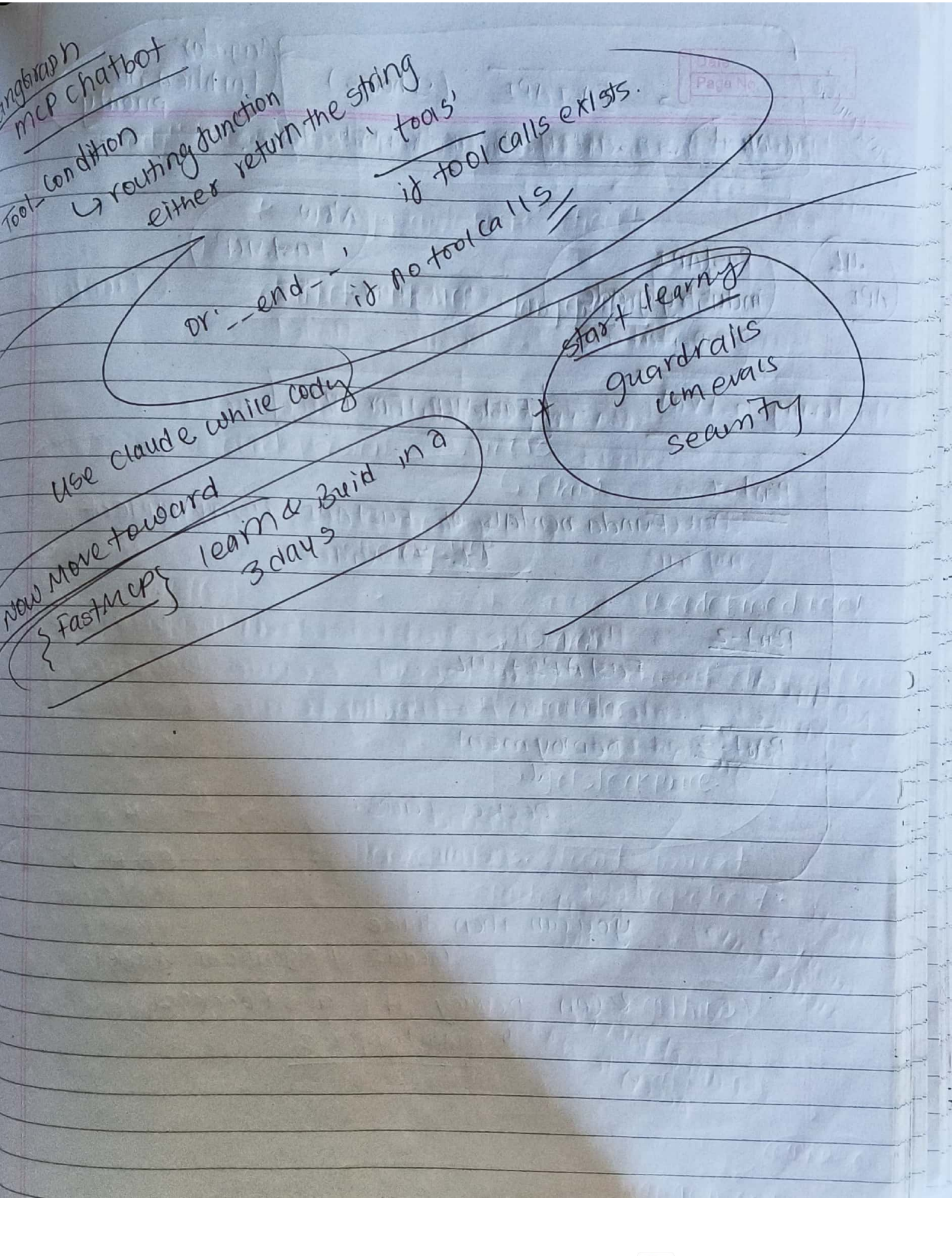
through langgraph

MCP server

FastMCP

async
↓
to do parallel execution

Scanned with OKEN Scanner


langgraph
mcp chatbot
tool_condition
↳ routing function
either return the string
'tools'
if tool calls exists.
or '__end__'
if no tool calls
Use claude while cody
start learning
+ guardrails
+ evals
+ security
now Move toward
{ FastMCP }
learn & Build in a
3 days